

Recording flow: instant

Mohanad Kandil

May 26, 2026

CONTENTS

Making the recording feel instant
The starting point
Under the hood, before
What we changed
End-of-session upload
A small organizational fix
How we measured
Results
Takeaway

Making the recording feel instant

What was slow in the voice-journaling flow, what we changed, and how we know it worked.

RE:SELF — Reflection-to-Action

The starting point

Tapping **Done** on a step used to leave the screen sitting there for two or three seconds before moving on. The freeze wasn't the network. The phone was doing too many things in a row before letting the UI move.

Under the hood, before

Every step ran four heavy operations back to back, all on the main thread:

1. Stopping the audio finalized the m4a file.
2. The motion-sensor buffer flushed to Firestore.
3. The local recording list was rewritten on disk.
4. The upload was queued.

On top of that, every tap on **Record** re-checked the microphone permission and probed the sensors, and the waveform held a three-second warm-up before it appeared.

What we changed

We let the UI move the moment you tap **Done**, and pushed everything heavy into the background.

The screen advances to the next prompt in a single frame. Finalizing the file, flushing the sensors, saving locally, and queueing the upload now run as a quiet pipeline behind it.

Permissions and the audio session are pre-warmed when the recording screen first mounts, so tapping **Record** costs nothing. The waveform shows from the first second.

Local saves write to an in-memory cache and persist to disk after the fact, so they're effectively free.

End-of-session upload

The final upload used to push recordings one after another. Now three go up at once. Same storage contract, same retry behavior — we just stopped waiting on each file before starting the next one.

A small organizational fix

Each five-step run now carries one **session id**. The timeline groups the recordings as *Session 1*, *Session 2* instead of ten loose entries scattered across the day. Older recordings without an id are grouped by time and step number so nothing looks orphaned.

How we measured

- **Per-step latency**: stopwatch on a real device, from the tap on *Done* to the next prompt becoming visible, averaged across ten runs.
- **Final upload**: end-to-end across five recordings on Wi-Fi, same network for before and after.
- **Build health**: `tsc --noEmit` and `expo-doctor` both green across the change, so no SDK or library swap is hiding the numbers.

Results

Metric	Before	After
Per-step transition	~2.4 s	instant (single frame)
Final upload (5 files, Wi-Fi)	~14 s	~5 s
Code path	—	unchanged
SDK / libraries	—	no swap

Takeaway

The user got the two seconds back they were losing on every step. Nothing exotic — we moved blocking work off the UI thread, cached what we kept reading, and stopped waiting on things that didn't need to be waited on.

Next: dynamic prompts (Plan D — multi-agent on-device).